

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf\_191a9b86-991\agent-ac05bc60188a1b0e4.jsonl`
- SHA-256: `71bd96197b2c8d0789081635559ff3302734788c0e9bf3384a30bba8dc7fabf0`
- Source modified: `2026-06-22T03:50:15+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_191a9b86-991`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T03:49:17.546Z)

Adversarially verify this P2c review finding by reading the cited code in C:\\Users\\avidu\\Projects\\khelsutra-p2c yourself (run `cd C:\\Users\\avidu\\Projects\\khelsutra-p2c && git diff origin/master...HEAD`). Is it a REAL defect to fix, or a false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: ally-ux
Severity(claimed): major
Title: role="dialog" on a non-blocking inline panel is misleading (no aria-modal, no focus trap, page stays interactive)
File: web/src/components/SetupWizard.tsx:120-124
Detail: The wizard is rendered inline inside the page section (App.tsx:242), not as an overlay: the header + LanguageSwitcher above it (App.tsx:227) stay fully interactive and the rest of the page remains in the DOM and tab order. Yet it is marked role="dialog" with no aria-modal, no initial-focus move, no focus trap, and no Escape-to-close. role="dialog" signals to AT users a modal context that captures focus and inerts the background ? none of which is true here. This is the inverse of the question's framing: for a genuinely non-blocking inline panel, role="dialog" is the misleading choice, not an acceptable one. role="region" with the existing aria-label would honestly describe what this is. (If a true modal is intended, it needs aria-modal="true" + focus trap + Escape + a backdrop ? but that contradicts the "inline panel" design in the file's own header comment.)
Fix: Switch role="dialog" to role="region" (keep aria-label={t("wizard.title")}), since the panel does not trap focus or block the page. Alternatively commit to a real modal: add aria-modal, move focus on open, trap focus, handle Escape.

2. assistant (2026-06-22T03:49:19.709Z)

I'll verify this finding by examining the actual code in the cited repository.

3. user (2026-06-22T03:49:23.847Z)

b67e758 chore(site): mirror wizard.* catalog keys into site/public/locales (copy:locales)
e6b9725 feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)

4. user (2026-06-22T03:49:24.062Z)

locales/da/common.json	32 ++++++
locales/en/common.json	32 ++++++
locales/es/common.json	32 ++++++
locales/hi/common.json	32 ++++++
locales/id/common.json	32 ++++++
locales/kn/common.json	32 ++++++
site/public/locales/da/common.json	32 ++++++
site/public/locales/en/common.json	32 ++++++
site/public/locales/es/common.json	32 ++++++

```

site/public/locales/hi/common.json      | 32 +++++
site/public/locales/id/common.json      | 32 +++++
site/public/locales/kn/common.json      | 32 +++++
web/src/App.tsx                          | 9 +-
web/src/api/client.ts                   | 6 ++
web/src/api/types.ts                     | 17 +++
web/src/components/SetupWizard.test.tsx | 115 ++++++
web/src/components/SetupWizard.tsx      | 179 ++++++
web/src/components/a4.touch.test.tsx    | 4 +
18 files changed, 713 insertions(+), 1 deletion(-)

```

5. assistant (2026-06-22T03:49:26.523Z)

Let me read the actual SetupWizard component and how it's rendered in App.tsx.

6. user (2026-06-22T03:49:27.478Z)

```

1      // First-run setup wizard (PACKAGING_PLAN.md P2c). Three quick steps ? AI key / compute
/ videos ?
2      // rendered from the engine's GET /api/capabilities (an honest probe of what THIS box
can do).
3      // Localized (en/kn/hi/es/da/id). The Gemini-key step is GUIDED setup, not auto-save:
the engine has
4      // no key-write endpoint yet, so we tell the user how to add the key (offline `motion`
works with none).
5      import { useEffect, useState } from "react";
6      import { useTranslation } from "react-i18next";
7      import { getCapabilities } from "../api/client";
8      import type { EngineCapabilities } from "../api/types";
9      import { log, newCorrelationId } from "../lib/log";
10
11     /** localStorage flag ? distinct from the language key ? so the wizard shows ONCE per
browser. */
12     export const WIZARD_DONE_KEY = "kheisutra.wizard.completed";
13
14     export function isWizardDone(): boolean {
15         try {
16             return localStorage.getItem(WIZARD_DONE_KEY) === "true";
17         } catch {
18             return false; // private-mode / no storage ? show it (harmless; user can skip)
19         }
20     }
21
22     const STEPS = 3;
23
24     function KeyStep({ caps }: { caps: EngineCapabilities | null }) {
25         const { t } = useTranslation();
26         const present = caps?.gemini_key_present === true;
27         return (
28             <div>
29                 <h3 className="font-semibold">{t("wizard.key.title")}</h3>
30                 {present ? (
31                     <p className="mt-2 font-medium text-green">{t("wizard.key.connected")}</p>
32                 ) : (
33                     <>
34                         <p className="mt-2 text-ink/80">{t("wizard.key.missing")}</p>
35                         <p className="mt-2 text-sm text-ink/70">{t("wizard.key.how")}</p>
36                         <a
37                             href="https://aistudio.google.com/apikey"
38                             target="_blank"
39                             rel="noreferrer"
40                             className="mt-2 inline-block text-sm font-medium text-green underline"
41                         >
42                             {t("wizard.key.getKey")}
43                         </a>

```

```

44         </>
45     })
46 </div>
47 );
48 }
49
50 function ComputeStep({ caps }: { caps: EngineCapabilities | null }) {
51     const { t } = useTranslation();
52     const ffmpegOk = caps?.ffmpeg?.ffmpeg === true && caps?.ffmpeg?.ffprobe === true;
53     const gpu = caps?.gpu?.nvidia_smi_present === true;
54     const detector = caps?.default_segmenter ?? "motion";
55     return (
56         <div>
57             <h3 className="font-semibold">{t("wizard.compute.title")}</h3>
58             <p className="mt-2 text-ink/80">{t("wizard.compute.local")}</p>
59             <ul className="mt-3 space-y-1 text-sm text-ink/70">
60                 <li>{t("wizard.compute.detector", { name: detector })}</li>
61                 <li>{ffmpegOk ? t("wizard.compute.ffmpegOk") :
t("wizard.compute.ffmpegMissing")}</li>
62                 <li>{gpu ? t("wizard.compute.gpu") : t("wizard.compute.noGpu")}</li>
63             </ul>
64         </div>
65     );
66 }
67
68 function VideosStep() {
69     const { t } = useTranslation();
70     return (
71         <div>
72             <h3 className="font-semibold">{t("wizard.videos.title")}</h3>
73             <p className="mt-2 text-ink/80">{t("wizard.videos.body")}</p>
74             <p className="mt-2 font-medium text-green">{t("wizard.videos.ready")}</p>
75         </div>
76     );
77 }
78
79 export function SetupWizard({ onClose }: { onClose: () => void }) {
80     const { t } = useTranslation();
81     const [step, setStep] = useState(0);
82     const [caps, setCaps] = useState<EngineCapabilities | null>(null);
83     const [loading, setLoading] = useState(true);
84     const [offline, setOffline] = useState(false);
85
86     useEffect(() => {
87         let alive = true;
88         const cid = newCorrelationId();
89         getCapabilities(cid)
90             .then((c) => {
91                 if (!alive) return;
92                 setCaps(c);
93                 setLoading(false);
94             })
95             .catch((err) => {
96                 if (!alive) return;
97                 // Engine not up yet ? don't assert false capabilities; offer a graceful skip.
98                 setOffline(true);
99                 setLoading(false);
100                 log("warn", "wizard.capabilities_unavailable", { cid, error: String(err) });
101             });
102         return () => {
103             alive = false;
104         };
105     }, []);
106
107     const finish = () => {

```

```

108     try {
109         localStorage.setItem(WIZARD_DONE_KEY, "true");
110     } catch {
111         /* private mode: still close (the wizard just reappears next visit) */
112     }
113     onClose();
114 };
115
116 const stepping = !loading && !offline; // the 3-step flow only when capabilities
loaded
117 const isLast = step === STEPS - 1;
118
119 return (
120     <section
121         role="dialog"
122         aria-label={t("wizard.title")}
123         className="mt-8 rounded-2xl border border-ink/10 bg-white p-6 shadow-sm"
124     >
125         <div className="flex items-center justify-between gap-3">
126             <h2 className="text-xl font-bold">{t("wizard.title")}</h2>
127             <button
128                 type="button"
129                 onClick={finish}
130                 className="min-h-11 rounded-lg px-3 text-sm text-ink/60 hover:text-ink"
131             >
132                 {t("wizard.skip")}
133             </button>
134         </div>
135         <p className="mt-1 text-sm text-ink/70">{t("wizard.intro")}</p>
136         {stepping && (
137             <p className="mt-1 text-xs font-medium text-ink/50">{t("wizard.step", { n: step
+ 1 })}</p>
138         )}
139
140         <div className="mt-5 min-h-[128px]" aria-live="polite">
141             {loading && <p className="text-ink/60">{t("wizard.checking")}</p>}
142             {!loading && offline && <p className="text-amber-700">{t("wizard.offline")}</p>}
143             {stepping && step === 0 && <KeyStep caps={caps} />}
144             {stepping && step === 1 && <ComputeStep caps={caps} />}
145             {stepping && step === 2 && <VideosStep />}
146         </div>
147
148         <div className="mt-6 flex items-center justify-between">
149             {stepping ? (
150                 <>
151                     <button
152                         type="button"
153                         disabled={step === 0}
154                         onClick={() => setStep((s) => Math.max(0, s - 1))}
155                         className="min-h-11 rounded-lg px-4 text-sm font-medium text-ink
disabled:opacity-40"
156                     >
157                         {t("wizard.back")}
158                     </button>
159                     <button
160                         type="button"
161                         onClick={() => (isLast ? finish() : setStep((s) => Math.min(STEPS - 1, s +
162                             1)))}
163                         className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
164                     >
165                         {isLast ? t("wizard.done") : t("wizard.next")}
166                     </button>
167                 </>
168             ) : (

```

```

168         <button
169             type="button"
170             onClick={finish}
171             className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
172         >
173             {t("wizard.done")}
174         </button>
175     )}
176 </div>
177 </section>
178 );
179 }
180

```

7. user (2026-06-22T03:49:27.713Z)

```

1     import { useEffect, useMemo, useState } from "react";
2     import { useTranslation, Trans } from "react-i18next";
3     import { useRallySelection } from "../hooks/useRallySelection";
4     import { RallyGrid } from "../components/RallyGrid";
5     import { QueryBar } from "../components/QueryBar";
6     import { TapFilters } from "../components/TapFilters";
7     import { SelectionFooter } from "../components/SelectionFooter";
8     import { LanguageSwitcher } from "../components/LanguageSwitcher";
9     import { CompileResult, type BuildState } from "../components/CompileResult";
10    import { IngestPanel } from "../components/IngestPanel";
11    import { SetupWizard, isWizardDone } from "../components/SetupWizard";
12    import { runQuery, type ParsedQuery } from "../lib/query";
13    import { floorRisk } from "../lib/floor";
14    import { reelFilename } from "../lib/format";
15    import { classifyError } from "../lib/errors";
16    import { errorText } from "../lib/errorText";
17    import { log, newCorrelationId } from "../lib/log";
18    import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
19    import type { EngineConfig, RallySegment } from "../api/types";
20
21    /** One compact, structured description of how a query was understood (for the trace
log). */
22    function parseTrace(q: ParsedQuery) {
23        return {
24            raw: q.raw,
25            recognized: q.recognized,
26            filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
27            sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
28            limit: q.limit ?? null,
29            unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
30        };
31    }
32
33    // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
34    // contract; only honest fields are surfaced. A real session loads from POST /api/query.
35    const DEMO_RALLIES: RallySegment[] = [
36        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
37        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
38        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
39        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
40        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },

```

```

41      { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
42    ];
43
44    type LoadState =
45      | { status: "demo" }
46      | { status: "loading" }
47      | { status: "loaded"; count: number }
48      | { status: "error"; error: string };
49
50    export default function App() {
51      const { t, i18n } = useTranslation();
52      // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
53      // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
54      // without a reload (it also rewrites ?video= so a refresh is stable).
55      const [videoId, setVideoId] = useState<string | null>(
56        () => new URLSearchParams(window.location.search).get("video"),
57      );
58
59      const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
60      const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
61      const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
62      const [reelName, setReelName] = useState("highlights");
63      const [build, setBuild] = useState<BuildState>({ status: "idle" });
64      // P2c: the first-run wizard shows once per browser, before the ingest panel, when
nothing's loaded.
65      const [wizardDone, setWizardDone] = useState(isWizardDone);
66
67      // Display order is separate from selection: a query can re-sort the cards while the
shared
68      // selection set (sel) stays the single source of truth for what lands in the reel.
69      const sel = useRallySelection(rallies, "all");
70
71      // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
72      useEffect(() => {
73        if (!videoId) return;
74        const cid = newCorrelationId();
75        log("info", "rallies.load_start", { cid, video_id: videoId });
76        // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
77        // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
78        setLoadState({ status: "loading" });
79        let alive = true;
80        listRallies(videoId, cid)
81          .then((rs) => {
82            if (!alive) return;
83            setRallies(rs);
84            setOrder(rs);
85            sel.apply(rs.map((r) => r.id));
86            setLoadState({ status: "loaded", count: rs.length });
87            log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
88          })
89          .catch((err) => {
90            if (!alive) return;
91            // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
92            // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
93            const e = classifyError(err);
94            setLoadState({ status: "error", error: errorText(t, e) });
95            log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });

```

```

96         });
97         return () => {
98             alive = false;
99         };
100         // videoId is stable for the page; sel.apply is a stable callback.
101         // eslint-disable-next-line react-hooks/exhaustive-deps
102     }, [videoId]);
103
104     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
105     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
106     const [cfg, setCfg] = useState<EngineConfig | null>(null);
107     useEffect(() => {
108         let alive = true;
109         getConfig()
110             .then((c) => alive && setCfg(c))
111             .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
112         return () => {
113             alive = false;
114         };
115     }, []);
116     const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
117
118     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
119     const ordinalOf = useMemo(() => {
120         const m = new Map<number, number>();
121         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
122         return (id: number) => m.get(id) ?? 0;
123     }, [rallies]);
124
125     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
126     const onBuild = async () => {
127         if (build.status === "building") return;
128         const cid = newCorrelationId();
129         const filename = reelFilename(reelName);
130         const ids = [...sel.selected];
131         if (!videoId) {
132             log("info", "build.no_video", { cid, selected: ids.length });
133             setBuild({
134                 status: "error",
135                 error: t("build.noVideo"),
136             });
137             return;
138         }
139         setBuild({ status: "building" });
140         log("info", "build.start", {
141             cid,
142             video_id: videoId,
143             filename,
144             selected: ids.length,
145             total_sec: Math.round(sel.totalSec),
146             below_floor: warning?.atRisk.length ?? 0,
147         });
148         try {
149             // Pass the active UI locale so the engine localizes the reel's branding watermark
(LW1).
150             const res = await compileReel(videoId, ids, filename, cid, i18n.resolvedLanguage
?? i18n.language);
151             const url = highlightUrl(videoId, filename);
152             setBuild({ status: "done", url, filename });
153             log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });

```

```

154         } catch (err) {
155             // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
156             // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
157             const status = err instanceof ApiError ? err.status : undefined;
158             const e = classifyError(err);
159             setBuild({ status: "error", error: errorText(t, e) });
160             log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
161         }
162     };
163
164     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
165     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
166     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
167     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
168         const cid = newCorrelationId();
169         log("info", "query.parse", { cid, source, ...parseTrace(q) });
170
171         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
172         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
173         if (q.unavailable.length > 0) {
174             log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
175         }
176
177         if (!q.recognized) {
178             log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
179             return; // empty / unparseable ? leave the user's curation untouched
180         }
181
182         try {
183             const { ordered, selectedIds } = runQuery(rallies, q);
184             setOrder(ordered);
185             sel.apply(selectedIds);
186             log("info", "query.apply", {
187                 cid,
188                 source,
189                 filters: q.filters.length,
190                 sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
191                 limit: q.limit ?? null,
192                 total: rallies.length,
193                 selected: selectedIds.length,
194                 selected_ids: selectedIds,
195                 dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
196             });
197             if (selectedIds.length === 0) {
198                 // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
199                 log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
200             }
201         } catch (err) {
202             // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
203             log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
204         }
205     };
206

```



```

207 // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
208 const onToggle = (id: number) => {
209   log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
210   sel.toggle(id);
211 };
212 const onBulk = (op: "select_all" | "clear" | "invert") => {
213   const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
214   log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
215   if (op === "select_all") sel.selectAll();
216   else if (op === "clear") sel.clearAll();
217   else sel.invert();
218 };
219
220 return (
221   <main className="min-h-screen bg-paper text-ink pb-24">
222     <header className="bg-ink text-white">
223       <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
224         <span className="text-xl font-extrabold tracking-tight">
225           Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
226         </span>
227         <LanguageSwitcher />
228       </div>
229     </header>
230
231     <section className="mx-auto max-w-5xl px-6 py-10">
232       <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
233         {t("app.badge")}
234       </span>
235       <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
236       <p className="mt-2 max-w-2xl text-ink/70">
237         <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
238       </p>
239
240       {/* P2c first-run wizard: before anything is loaded, walk a new user through AI-
key / compute /
241         videos (from /api/capabilities). Shows once per browser; then the ingest
panel takes over. */}
242       {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}
243
244       {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
245         path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
246       {!videoId && wizardDone && (
247         <IngestPanel
248           onLoad={id => {
249             setVideoId(id);
250             window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
251           }}
252         />
253       )}
254
255       <ModeBanner load={loadState} videoId={videoId} />
256
257       <div className="mt-6">
258         <QueryBar onApply={onApplyQuery} />
259         {/* A9: localized tap controls for users who can't type the English query
syntax. */}

```

```

260         <TapFilters onApply={onApplyQuery} />
261     </div>
262
263     <div className="mt-6 flex flex-wrap gap-2 text-sm">
264         <button type="button" onClick={() => onBulk("select_all")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.selectAll")}</button>
265         <button type="button" onClick={() => onBulk("clear")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
266         <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
267     </div>
268
269     <div className="mt-5">
270         <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
271     </div>
272
273     <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
274 </section>
275
276 <SelectionFooter
277     count={sel.count}
278     totalSec={sel.totalSec}
279     reelName={reelName}
280     onReelNameChange={setReelName}
281     onBuild={onBuild}
282     building={build.status === "building"}
283     warning={warning}
284 />
285 </main>
286 );
287 }
288
289 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
290     if (load.status === "demo") {
291         return (
292             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
293                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
294             </p>
295         );
296     }
297     if (load.status === "loading") {
298         return (
299             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
300                 <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
301             </p>
302         );
303     }
304     if (load.status === "error") {
305         return (
306             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
307                 <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
308             </p>
309         );
310     }
311     return (
312         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
313             <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />

```

```

314         </p>
315     );
316 }
317

```

8. assistant (2026-06-22T03:49:34.125Z)

The finding's structural claims check out against the code. Let me verify the precise rendering context and that the finding's description of the surrounding interactive elements is accurate.

The finding cites `SetupWizard.tsx:120-124` for `role="dialog" aria-label={t("wizard.title")}` ? confirmed (lines 120-124). It cites `App.tsx:242` for inline rendering ? confirmed: `{!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}` is rendered inline inside the `<section>`, not in any portal/overlay. It cites `App.tsx:227` for the `LanguageSwitcher` in the header above it ? confirmed (line 227), and the header sits above the section, fully in the DOM and tab order.

Let me confirm the key technical assertions: no `aria-modal`, no focus trap, no Escape handler, no backdrop/portal, and that the file's own header comment frames this as an inline panel.

9. user (2026-06-22T03:49:36.156Z)

No matches found

10. user (2026-06-22T03:49:36.522Z)

```

1      // @vitest-environment jsdom
2      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
3      import { render, screen, fireEvent, cleanup, waitFor } from "@testing-library/react";
4      import i18n from "../i18n/index";
5      import { SetupWizard, WIZARD_DONE_KEY, isWizardDone } from "../SetupWizard";
6      import type { EngineCapabilities } from "../api/types";
7
8      function stubCapabilities(caps: EngineCapabilities | "reject") {
9          vi.stubGlobal(
10              "fetch",
11              vi.fn(async (url: string | URL) => {
12                  const u = typeof url === "string" ? url : url.toString();
13                  if (u.endsWith("/api/capabilities")) {
14                      if (caps === "reject") throw new TypeError("Failed to fetch");
15                      return { ok: true, status: 200, json: async () => caps } as unknown as Response;
16                  }
17                  return { ok: false, status: 404, text: async () => "" } as unknown as Response;
18              }),
19          );
20      }
21
22      const LIGHT: EngineCapabilities = {
23          gemini_key_present: false,
24          default_segmenter: "motion",
25          ffmpeg: { ffmpeg: false, ffprobe: false },
26          gpu: { nvidia_smi_present: false },
27      };
28      const READY: EngineCapabilities = {
29          gemini_key_present: true,
30          default_segmenter: "gemini",
31          ffmpeg: { ffmpeg: true, ffprobe: true },
32          gpu: { nvidia_smi_present: true },
33      };
34
35      beforeEach(async () => {
36          window.localStorage.clear();
37          await i18n.changeLanguage("en");
38      });

```

```

39     afterEach(() => {
40         cleanup();
41         vi.restoreAllMocks();
42         vi.unstubAllGlobals();
43     });
44
45     describe("SetupWizard", () => {
46         it("checks capabilities, then step 1 guides a missing Gemini key (offline-first
default)", async () => {
47             stubCapabilities(LIGHT);
48             render(<SetupWizard onClose={() => {}} />);
49             expect(screen.getByText(/Checking what this device can do/)).toBeTruthy();
50             expect(await screen.findByText(/built-in motion detector/)).toBeTruthy();
51             expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
52             expect(screen.getByText(/Step 1 of 3/)).toBeTruthy();
53         });
54
55         it("walks all 3 steps from real capabilities; Back returns; Done sets the flag +
closes", async () => {
56             stubCapabilities(LIGHT);
57             const onClose = vi.fn();
58             render(<SetupWizard onClose={onClose} />);
59             await screen.findByText(/built-in motion detector/);
60
61             fireEvent.click(screen.getByText("Next")); // ? compute
62             expect(screen.getByText("Detector: motion")).toBeTruthy();
63             expect(screen.getByText(/ffmpeg not found/)).toBeTruthy();
64             expect(screen.getByText(/No GPU needed/)).toBeTruthy();
65
66             fireEvent.click(screen.getByText("Back")); // ? key
67             expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
68
69             fireEvent.click(screen.getByText("Next")); // ? compute
70             fireEvent.click(screen.getByText("Next")); // ? videos
71             expect(screen.getByText(/You're all set/)).toBeTruthy();
72
73             fireEvent.click(screen.getByText("Get started")); // Done on last step
74             expect(onClose).toHaveBeenCalledTimes(1);
75             expect(window.localStorage.getItem(WIZARD_DONE_KEY)).toBe("true");
76         });
77
78         it("reflects a connected key + a GPU/ffmpeg-ready box", async () => {
79             stubCapabilities(READY);
80             render(<SetupWizard onClose={() => {}} />);
81             expect(await screen.findByText(/Gemini is connected/)).toBeTruthy();
82             fireEvent.click(screen.getByText("Next"));
83             expect(screen.getByText("Detector: gemini")).toBeTruthy();
84             expect(screen.getByText(/Video tools \(\ffmpeg\) ready/)).toBeTruthy();
85             expect(screen.getByText(/A GPU was detected/)).toBeTruthy();
86         });
87
88         it("Skip closes + sets the flag without finishing the steps", async () => {
89             stubCapabilities(LIGHT);
90             const onClose = vi.fn();
91             render(<SetupWizard onClose={onClose} />);
92             await screen.findByText(/built-in motion detector/);
93             fireEvent.click(screen.getByText("Skip"));
94             expect(onClose).toHaveBeenCalledTimes(1);
95             expect(isWizardDone()).toBe(true);
96         });
97
98         it("degrades gracefully when the engine is unreachable (offline) ? message + a Done
escape", async () => {
99             stubCapabilities("reject");
100             const onClose = vi.fn();

```

```

101     render(<SetupWizard onClose={onClose} />);
102     expect(await screen.findByText(/Couldn't reach KhelSutra/)).toBeTruthy();
103     // No step content / step indicator when offline.
104     expect(screen.queryByText(/Step 1 of 3/)).toBeNull();
105     fireEvent.click(screen.getByText("Get started"));
106     await waitFor(() => expect(onClose).toHaveBeenCalled());
107     expect(isWizardDone()).toBe(true);
108   });
109
110   it("isWizardDone reads the localStorage flag", () => {
111     expect(isWizardDone()).toBe(false);
112     window.localStorage.setItem(WIZARD_DONE_KEY, "true");
113     expect(isWizardDone()).toBe(true);
114   });
115 });
116

```

11. assistant (2026-06-22T03:49:57.057Z)

All technical claims in the finding are verified against the actual code:

1. `**role="dialog"` with no `aria-modal` ? confirmed. Grep for `aria-modal` returns no matches. The `<section role="dialog" aria-label={t("wizard.title")}>` (lines 120-122) has no `aria-modal`.
2. `**No focus trap, no initial-focus move, no Escape-to-close` ? confirmed. Grep for `useRef`, `focus`, `onKeyDown`, `Escape`, `inert`, `trapFocus` returns no matches. The component is purely declarative with no focus management.
3. `**No backdrop / not an overlay` ? confirmed. Grep for `createPortal`, `fixed inset`, `backdrop` returns no matches. It's a plain `<section>` with `className="mt-8 rounded-2xl ..."` ? flow-positioned inline, not portaled or fixed.
4. `**Rendered inline inside the page section` ? confirmed at `App.tsx:242`: `{!videoId && !wizardDone && <SetupWizard onClose={...} />}` sits directly inside `<section className="mx-auto max-w-5xl ...">`, with the `<header>` containing `LanguageSwitcher` (line 227) above it, all in normal DOM/tab order.
5. `**The file's own header comment frames it as a non-blocking inline panel` ? confirmed. Lines 1-4 describe "Three quick steps... rendered from the engine's GET /api/capabilities" and the design intent is a guided, skippable inline panel (Skip/Done buttons, shows once per browser). Nothing indicates a modal interaction model.

Now my adversarial assessment of whether this is a REAL defect worth fixing:

`**The finding is technically accurate, but the severity is overstated.**` The core ARIA-semantics observation is correct: `role="dialog"` (without `aria-modal`) does carry an implication to AT users that they've entered a dialog context, and the standard pattern pairs `role="dialog"` with focus management. Per the WAI-ARIA Authoring Practices, a dialog should manage focus. Here there is no focus management at all, and the background genuinely stays interactive. So `role="region"` (keeping the existing `aria-label`) would be the more honest mapping for what this actually is. This is a legitimate, low-risk, correct fix.

However, weighing it down from "major":

- `role="dialog"` *without* `aria-modal="true"` is the non-modal dialog flavor, which by spec does NOT promise the background is inert. So the claim that it "signals... a modal context that captures focus and inerts the background" overstates what a bare `role="dialog"` guarantees ? a non-modal dialog legitimately leaves the background reachable. The real, narrower defect is the missing focus management (a dialog of any kind should move focus into itself on open), plus the absence of a labelled relationship that `aria-labelledby` to the `<h2>` would give.
- Real-world impact is bounded: the panel still has a visible, reachable heading, Skip and Done/Next controls, it's keyboard-operable, and content is announced via `aria-live`. No user is trapped or blocked. The harm is a mild semantic mismatch (a screen-reader user is told "dialog" for something that behaves like a region), not a broken or unusable flow.

This lands as a **minor** accessibility correctness issue: a real, well-located semantic mismatch with a trivial, safe fix (`role="region"`), but not a blocker or major-severity barrier, and the finding's framing of dialog-implies-inert-background is somewhat stronger than the spec supports.

12. user (2026-06-22T03:50:06.796Z)

Structured output provided successfully

13. assistant (2026-06-22T03:50:15.449Z)

I've completed the adversarial verification.

The finding is **REAL** but **minor** (not the claimed "major").

All cited code facts hold up against the actual diff in
`C:\Users\avidu\Projects\kheIsutra-p2c\web\src\components\SetupWizard.tsx:120-124`` and
`C:\Users\avidu\Projects\kheIsutra-p2c\web\src\App.tsx:242``: the panel is `role="dialog"` with no `aria-modal`, no focus trap, no initial focus move, no Escape handler, no portal/backdrop, and it's rendered inline with the header/LanguageSwitcher staying fully interactive above it. The suggested fix (`role="region"` keeping the `aria-label`) is correct.

I downgraded from major to minor because (a) one premise is overstated ? a bare `role="dialog"` **without** `aria-modal="true"` is the non-modal flavor that does NOT promise an inert background per WAI-ARIA, so the "signals a modal context that inerts the background" framing is stronger than spec supports; and (b) real impact is bounded ? the panel has a visible reachable heading, keyboard-operable Skip/Next/Done controls, and `aria-live` announcements, so no user is trapped or blocked. It's a genuine semantic-honesty mismatch with a trivial safe fix, not a usability barrier.